DBMS Viva Questions & Answers

COMPLETE SQL & PL/SQL EXAM REFERENCE GUIDE

Subject: Database Management Systems
Topics: SQL Subsets, Keys, ER Diagrams, PL/SQL

Q1 SQL Command Subsets (DDL, DML, DCL, TCL)

SQL commands are grouped into functional subsets based on whether they modify database structure, data, security, or transaction lifecycles:

- **DDL (Data Definition Language):** Defines and alters the database schema and structure. These operations auto-commit immediately.
  - `CREATE` : Creates a new table, view, or database object.
  - `ALTER` : Modifies an existing database structure.
  - `DROP` : Deletes an entire database object and its structure.
  - `TRUNCATE` : Removes all rows from a table while preserving structure.
  - `RENAME` : Renames a table or database object.
- **DML (Data Manipulation Language):** Manages, modifies, and queries the data stored inside database objects.
  - `INSERT` : Adds new rows into a table.
  - `UPDATE` : Modifies existing rows based on conditions.
  - `DELETE` : Removes specific rows based on a condition.
  - `SELECT` *(often classified as DQL - Data Query Language)*: Retrieves records from tables.
- **DCL (Data Control Language):** Controls permissions, privileges, and user access rights.
  - `GRANT` : Gives privileges (e.g., `SELECT` , `UPDATE`) to database users or roles.
  - `REVOKE` : Withdraws previously granted privileges.
- **TCL (Transaction Control Language):** Manages atomic units of work (transactions) made by DML statements.
  - `COMMIT` : Permanently saves all changes made in the current transaction.
  - `ROLLBACK` : Undoes all modifications made since the last `COMMIT` or to a specified `SAVEPOINT` .
  - `SAVEPOINT` : Sets a temporary marker within a transaction to allow partial rollbacks.

Q2 What is a Subquery?

A **subquery** (or nested/inner query) is a `SELECT` statement embedded inside another SQL statement (the outer query), such as inside a `WHERE` , `HAVING` , `FROM` , or `SELECT` clause.

- **Single-row Subquery:** Returns one row (evaluated using operators like `=` , `>` , `<`).

```
SELECT emp_name, salary
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

- **Multi-row Subquery:** Returns multiple rows (evaluated using operators like `IN` , `ANY` , `ALL`).

```
SELECT emp_name
FROM employees
WHERE dept_id IN (SELECT dept_id FROM departments WHERE location = 'Chicago');
```

- **Correlated Subquery:** An inner query that references a column from the outer query table, executing once for each row evaluated by the outer query.

Q3 Comparative Breakdowns: DELETE vs. TRUNCATE vs. DROP & TCL

DELETE vs. TRUNCATE vs. DROP

Feature	`DELETE`	`TRUNCATE`	`DROP`
Command Category	DML	DDL	DDL
Target Scope	Specific rows or all rows	All rows in the table	Entire table (data + structure)
`WHERE` Clause	Supported	Not supported	Not supported
Speed	Slow (logs row-by-row deletions)	Very fast (deallocates data pages)	Immediate (removes metadata)
Rollback Capability	Yes (can be rolled back in a transaction)	No / DBMS-dependent (auto-commits)	No (auto-commits immediately)
Triggers	Fires `ON DELETE` triggers	Does not fire `DELETE` triggers	Does not fire triggers
Schema/Structure	Retained	Retained	Destroyed completely
Auto-increment ID	Does not reset sequence/identity	Resets sequence/identity to seed	Destroyed along with table

ROLLBACK vs. TRUNCATE vs. SAVEPOINT

- **`ROLLBACK` (TCL):** An operation that cancels all pending DML changes (`INSERT` , `UPDATE` , `DELETE`) executed during the current active transaction, restoring the database to its last committed state.
- **`TRUNCATE` (DDL):** An operation that completely purges data from a table and frees allocated disk storage without dropping the table definition. It cannot be targeted at specific rows.
- **`SAVEPOINT` (TCL):** A transaction bookmark created using `SAVEPOINT savepoint_name;` . It allows partial reversal using `ROLLBACK TO savepoint_name;` without undoing the entire transaction.

Q4 Use of the ALTER Command

The `ALTER` command modifies an existing table structure without dropping or recreating it. Common use cases and syntax:

1. **Add a new column:**

```
ALTER TABLE employees ADD email VARCHAR(100);
```

2. **Modify column data type or constraints:**

```
ALTER TABLE employees MODIFY salary DECIMAL(12, 2) NOT NULL;
-- PostgreSQL / SQL Server variant:
-- ALTER TABLE employees ALTER COLUMN salary TYPE DECIMAL(12, 2);
```

3. **Drop a column:**

```
ALTER TABLE employees DROP COLUMN email;
```

4. **Add or remove constraints:**

```
ALTER TABLE employees ADD CONSTRAINT chk_sal CHECK (salary > 0);
ALTER TABLE employees DROP CONSTRAINT chk_sal;
```

Q5 How to Rename a Table & Syntax

Syntax varies depending on the database engine:

- **Standard ANSI SQL / Oracle / PostgreSQL / SQLite:** `ALTER TABLE old_table_name RENAME TO new_table_name;`
- **MySQL:** `RENAME TABLE old_table_name TO new_table_name;`
- **Microsoft SQL Server (T-SQL):** `EXEC sp_rename 'old_table_name', 'new_table_name';`

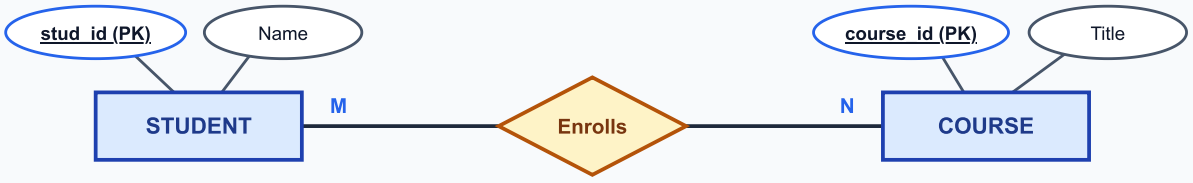
Q6 Primary Key (PK) vs. Foreign Key (FK)

- **Primary Key (PK):** A column or combination of columns that uniquely identifies every row in a table. Cannot contain `NULL` values, only one primary key constraint is permitted per table, and it automatically creates a unique index.
- **Foreign Key (FK):** A column (or set of columns) that references the primary key or unique key of another table (or the same table in self-referencing relationships). Establishes **referential integrity**, can accept `NULL` values (unless restricted by `NOT NULL`), and multiple foreign keys can exist in a single table.

```
CREATE TABLE departments (
    dept_id INT PRIMARY KEY,
    dept_name VARCHAR(50) NOT NULL
);
CREATE TABLE employees (
    emp_id INT PRIMARY KEY,              -- PK
    emp_name VARCHAR(50) NOT NULL,
    dept_id INT,                         -- FK referencing departments
    FOREIGN KEY (dept_id) REFERENCES departments(dept_id)
);
```

Q7 Entity-Relationship (ER) Modeling & Diagram Notations

An **ER Diagram** visually maps real-world entities, their properties, and how they relate inside a database.



Standard ER Diagram Components (Chen Notation: Entities, Key Attributes, Relationship & M:N Cardinality)

Core ER Components (Chen Notation)

- **Rectangle:** Entity (e.g., `Student`, `Course`).
- **Double Rectangle:** Weak Entity (depends on an identifying owner entity; cannot exist independently).
- **Ellipse/Oval:** Attribute (e.g., `Name`, `DOB`).
- **Underlined text in Ellipse:** Key Attribute (Primary Key).
- **Double Ellipse:** Multivalued Attribute (e.g., `phone_numbers`).
- **Dashed Ellipse:** Derived Attribute (e.g., `Age` calculated from `DOB`).
- **Diamond:** Relationship connecting entities (e.g., `Enrolls`).

Relationship Cardinality Types

- **One-to-One (1:1):** One record in Table A is associated with at most one record in Table B (e.g., `Citizen` and `Passport`).
- **One-to-Many (1:N):** One record in Table A associates with multiple records in Table B (e.g., `Department` and `Employees`).
- **Many-to-Many (M:N):** Multiple records in Table A associate with multiple records in Table B (e.g., `Students` and `Courses`). Requires a junction/bridge table in relational implementations.

Q8 PL/SQL Architecture, Program Blocks & Delimiters

PL/SQL (Procedural Language for SQL) extends SQL by adding control structures, loops, condition handling, and variables.

General Structure: `DECLARE`, `BEGIN`, `EXCEPTION`, `END`

A standard anonymous PL/SQL block consists of three sections:

```
-- 1. Declaration Section (Optional): Defines variables, cursors, and constants
DECLARE
    v_emp_name employees.emp_name%TYPE;
    v_salary   employees.salary%TYPE;

-- 2. Execution Section (Mandatory): Contains procedural logic and SQL commands
BEGIN
    SELECT emp_name, salary
    INTO v_emp_name, v_salary
    FROM employees
    WHERE emp_id = 101;

    DBMS_OUTPUT.PUT_LINE('Employee: ' || v_emp_name || ' | Salary: ' || v_salary);

-- 3. Exception Handling Section (Optional): Catches runtime errors
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('No employee found with the given ID.');
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('An unexpected error occurred: ' || SQLERRM);

-- Block Closure (Mandatory)
END;
/
```

Delimiters in PL/SQL

- `;` **(Semicolon):** Terminates individual statements inside the block.
- `:=` **(Assignment):** Assigns a value to a variable (e.g., `v_counter := 1;`).
- `||` **(Concatenation):** Joins strings together.
- `--` **(Double Hyphen):** Single-line comment.
- `/* ... */`: Multi-line comment.
- `/` **(Forward Slash on a new line):** Instructs SQL*Plus or SQL developer tools to send the entire block buffer to the Oracle engine for compilation and execution.

Q9 Data Types: SQL vs. PL/SQL

PL/SQL supports all standard SQL data types, while introducing procedural types like boolean flags, binary integers, and anchored attribute references:

Category	SQL Data Types	PL/SQL-Specific Data Types
Character/String	`CHAR(n)`, `VARCHAR2(n)`, `CLOB`	Standard types plus `LONG`, string subtypes
Numeric	`NUMBER(p, s)`, `INT`, `FLOAT`	`PLS_INTEGER`, `BINARY_INTEGER` (faster arithmetic in memory)
Date/Time	`DATE`, `TIMESTAMP`, `INTERVAL`	Identical date/time structures
Logical/Boolean	*No native standalone BOOLEAN column type in standard Oracle SQL*	`BOOLEAN` (accepts `TRUE`, `FALSE`, `NULL`)
Anchored Types	Not available in SQL DDL	`%TYPE` (inherits data type from specified table